Foundations and Formalism of Reinforcement Learning

A Structured Study Guide — From Physical Intuition to Mathematical Formalism

Machine Learning Specialization Companion | Markov Decision Processes, Returns & Policies

Meta-Analysis • Introduction • Helicopter • Reward & Puppy • Applications • Mars Rover • The Return • Policy • MDP • Interview Prep • Tailored Report

Part 1 — Meta-Analysis: The Tutor's Pedagogical Technique

This guide builds a rigorous understanding of Reinforcement Learning (RL) from first principles — transitioning from physical examples and biological analogies to the mathematical formalism of Markov Decision Processes (MDPs), returns, and policy optimization.

Before diving into the technical content, it is instructive to analyze *how* these concepts are taught. The lecture employs a deliberate, highly effective learning loop:

1. **Physical Mechanics to High-Dimensional Complexity** — The tutor opens with the Stanford autonomous helicopter. Starting with a complex, 3D physical system immediately engages curiosity while highlighting why traditional supervised learning fails under ambiguity.
2. **Behavioral and Biological Grounding** — To make the "reward function" intuitive, the tutor pivots to training a puppy with positive and negative feedback ("good dog", "bad dog"), shifting the framing to basic behaviorism: telling the agent *what* to do rather than *how* to do it.
3. **The Minimalist Grid-World Transition** — Having established the core tension, the tutor strips away physical noise (wind, gyroscopes, continuous space) and introduces a simplified, discrete model: the 6-state Mars Rover.
4. **Step-by-Step Mathematical Grounding** — Working directly on the 6-state grid, the tutor introduces the Return (G_t), the discount factor (γ), and the Policy (π) — walking through concrete arithmetic rather than abstract proofs.
5. **Formalization (The MDP Framework)** — Once the mechanics are internalized, the tutor reveals that this framework is mathematically formalized as a Markov Decision Process, then maps the helicopter and chess examples back onto this exact structure.

This teaching rhythm — **Grip (Helicopter)** → **Intuition (Puppy)** → **Abstraction (Mars Rover)** → **Calculation (Returns)** → **Formalization (MDP)** — ensures you never learn an equation without first understanding its physical or behavioral motivation.

1. Introduction: RL as a Core Pillar of ML

Motivation & Intuition

While supervised and unsupervised learning dominate commercial applications today, Reinforcement Learning represents a fundamental **third pillar** of machine learning. It is backed by intensive, rapidly progressing research that unlocks autonomous capabilities beyond static pattern recognition.

Core Concept

RL addresses problems where an agent must learn to make a sequence of decisions in an environment to achieve a long-term goal.

Tutor's Aside

> *"RL is not very widely applied in commercial applications yet today" compared to supervised learning — but it remains a crucial pillar with massive future potential.*

2. The Autonomous Helicopter Example

Motivation

Imagine being handed the keys to a 32-pound, instrumented Stanford autonomous helicopter (onboard computer, GPS, accelerometers, gyroscopes, magnetic compass) and asked to write a program to fly it.

The Continuous Control Loop

The joysticks on a radio-controlled transmitter must be adjusted **10 times per second** based on the helicopter's speed, orientation, and position. Radio-controlled helicopters are significantly harder to balance and fly than quadrotor drones.

The Failure of Supervised Learning

- One might frame this as supervised learning: map the state s (feature x) to joystick movements a (target label y) using human pilot demonstrations.
- *Why it fails*: In flight, the "correct" action is highly **ambiguous**. Should you move the stick a fraction of a millimeter, or a lot more? Because there is no single "ideal" action y for a given state x, collecting a clean, deterministic dataset of (x, y) pairs is virtually impossible.

The RL Alternative

Instead of telling the helicopter exactly *how* to fly at each millisecond, we let it fly and provide feedback when it succeeds or fails. Using this approach, the Stanford helicopter successfully learned complex aerobatic stunts, including **flying upside down**.

3. The Reward Function Analogy (The Puppy)

Motivation

How do we tell a system what is "good" and "bad" without explicitly dictating its actions?

The Analogy

Think of training a puppy. You cannot demonstrate to a puppy how to sit or behave by physically forcing or explaining it. Instead, you let the puppy act: when it does something good, you say "good dog"; when it does something bad, you say "bad dog." Over time, the puppy self-adjusts to maximize positive feedback and minimize negative feedback.

The Reinforcement Learning Analogy

- Good behavior (flying steady) → **"Good helicopter"** (e.g., reward of +1 per second).
- Bad behavior (crashing) → **"Bad helicopter"** (e.g., a massive negative reward of –1,000).

The Golden Rule of RL

Specify *what* to do, not *how* to do it. This gives the learning algorithm the ultimate flexibility to find optimal, sometimes highly creative, strategies that a human designer would never have conceived.

4. Robotic Dogs & Real-World Applications

The Robot Dog

The tutor shares a video of a robotic quadruped dog trained via RL, whose goal is to cross a variety of highly irregular obstacles. The robot is rewarded simply for making progress toward the left of the screen.

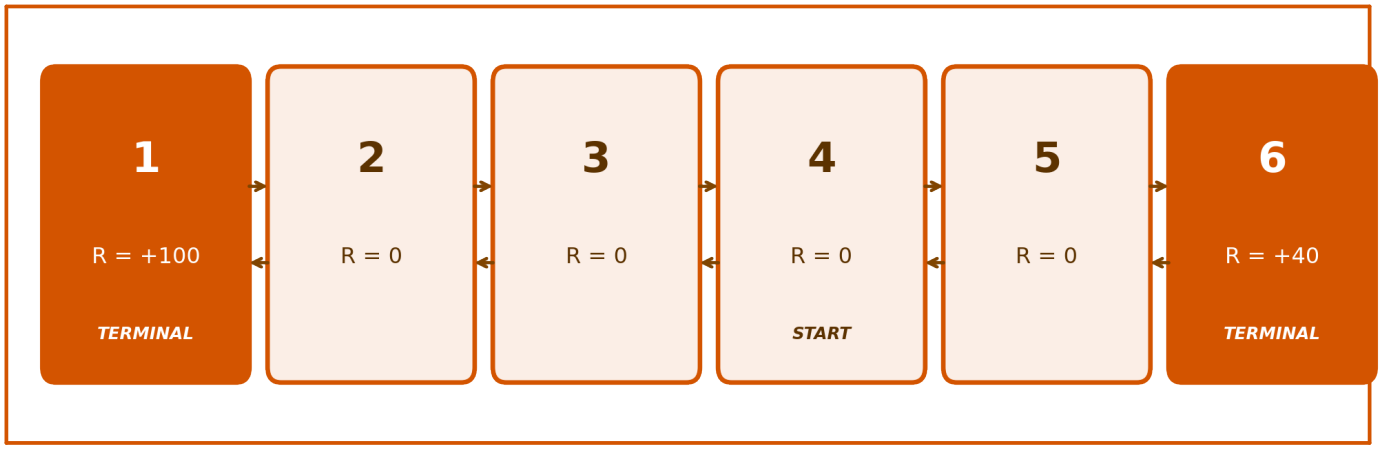
> *"I really don't know how to tell it what's the best way to place its legs" — yet RL figured out foot placement and climbing mechanics completely automatically.*

Broad Industry Applications

- **Lunar Lander** — landing a spacecraft safely in simulation.
- **Factory Optimization** — arranging equipment, schedules, and workflows to maximize efficiency and throughput.
- **Financial Stock Trading & Execution** — sequencing a large order (e.g., selling 1 million shares over several days) to avoid dumping stock all at once and obtain the best overall execution price.
- **Game Playing** — checkers, chess, bridge, Go, and various video games.

5. Formalizing RL: The Mars Rover Example

To transition from physical robots to mathematical formalism, the tutor introduces a simplified world inspired by the Mars Rover (developed with Stanford Prof. Emma Brunskill and Jagriti Agrawal, who has written code actively controlling the real Mars Rover).



The 6-state Mars Rover world. States 1 and 6 are terminal; the rover starts in State 4.

States (S)

The rover's position is modeled as 6 discrete states, numbered 1 to 6. In this example, the rover starts in state 4.

Actions (A)

At each step, the rover can choose one of two actions: **Go Left** or **Go Right**.

The Reward Function R(s)

- State 1 has extremely valuable scientific interest (e.g., a rich mineral sampling site). Its reward is **+100**.
- State 6 has moderate scientific interest. Its reward is **+40**.
- States 2, 3, 4, and 5 contain no scientific interest; their reward is **0**.

Terminal States

States 1 and 6 are terminal states. Once the rover enters either state, the “day ends” (fuel or time runs out). The rover receives the reward of that state, but no further actions can be taken and no additional rewards can be earned.

The State Transition Loop

At every step, the system sits in state *s*, selects action *a*, receives a reward R(s) associated with that current state, and transitions to a new state *s′*. **Important caveat**: the reward R(s) is tied to the state *s* you *start* the step in, not the state you arrive at. For example, taking action “Go Left” from state 4 yields a reward of 0 (the reward of state 4) and transitions you to state 3.

6. The Return in RL

Motivation & Intuition

If you have a $5 bill lying right at your feet, or you can walk 30 minutes across town to pick up a $10 bill, which do you choose? Although $10 is more than $5, the effort and time delay of the walk make the immediate $5 bill highly attractive. The **Return** mathematically formalizes this trade-off: rewards received sooner are more attractive than rewards that take a long time to get.

Mathematical Formulation

The Return is the sum of rewards weighted by the discount factor γ (Gamma), a number slightly less than 1 (typically 0.9, 0.99, or 0.999):

$$\text{Return} = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \ldots + \gamma^{T-1} R_T$$

- R_t is the reward received at step *t*.
- γ makes the algorithm **impatient**: it gives 100% credit to immediate rewards, but discounts future rewards by compounding powers of γ.

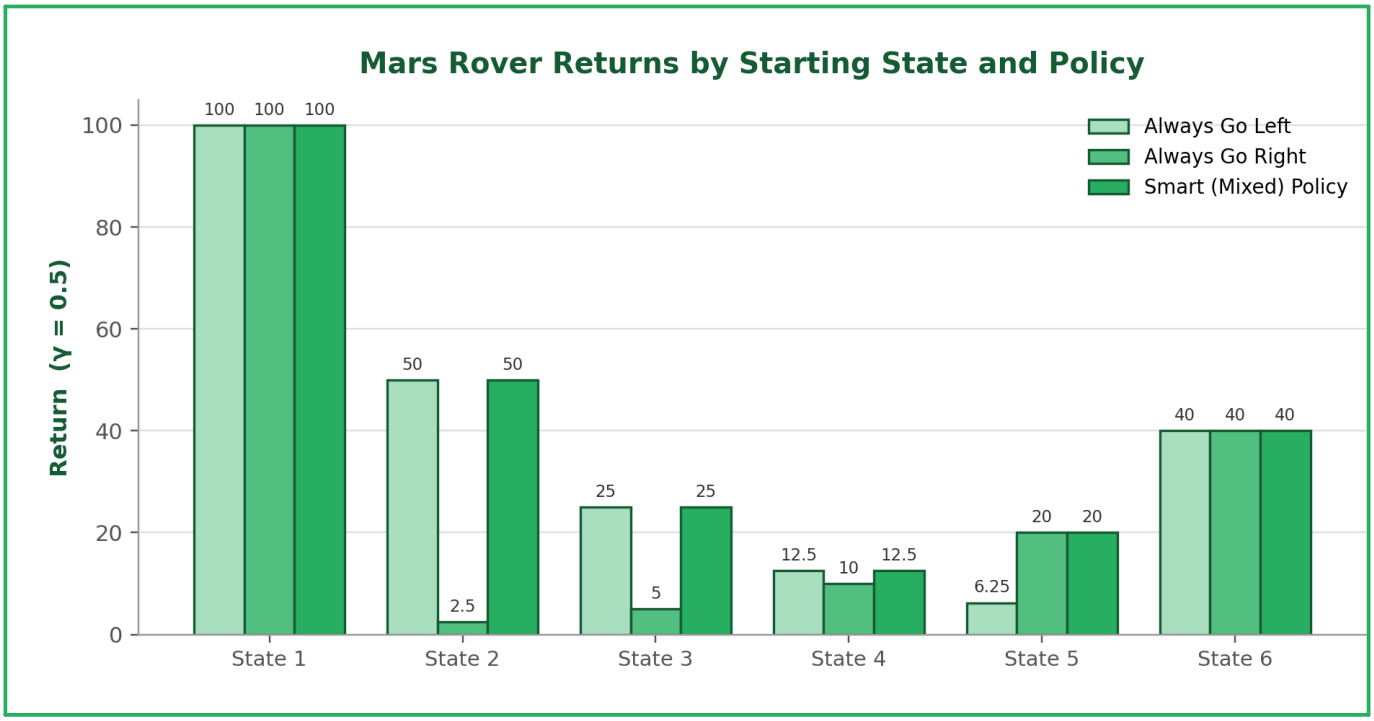
Financial Analogy

In financial applications, γ represents the **interest rate** or the time-value of money. A dollar today is worth more than a dollar tomorrow because you can put it in the bank, earn interest, and have more than a dollar in the future.

Working Out Returns (with $\gamma = 0.5$)

To heavily discount the future for clear arithmetic, the tutor uses $\gamma = 0.5$ as a running example. The table below compares the return from each starting state under three policies.

Starting State	Always Go Left	Always Go Right	Smart (Mixed) Policy
State 1 (terminal)	100	100	100
State 2	50	2.5	50
State 3	25	5	25
State 4 (start)	12.5	10	12.5
State 5	6.25	20	20
State 6 (terminal)	40	40	40

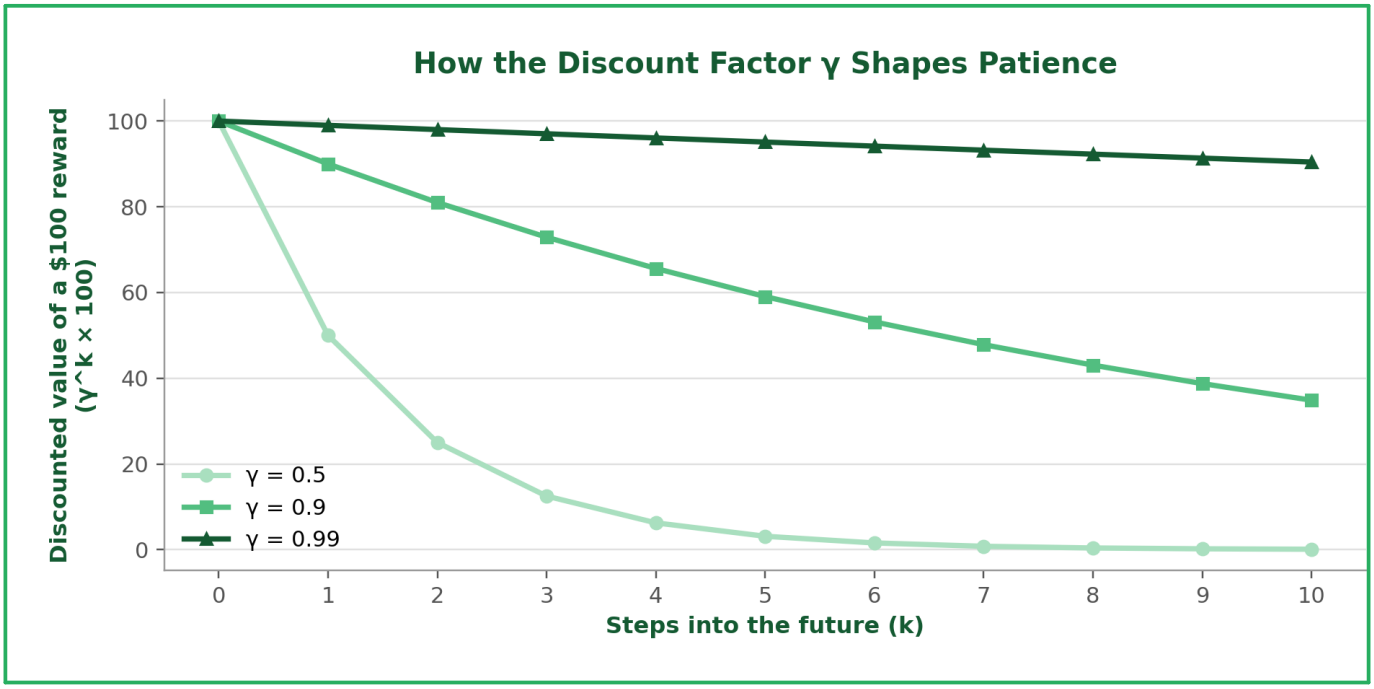


Returns for each starting state under the three candidate policies (γ = 0.5).

Looking at the returns from State 4: Always Go Left yields **12.5**; Always Go Right yields **10**. Going left is superior because the significantly larger terminal reward (100 vs 40) easily compensates for the extra step required to reach it.

Case C: A Mixed “Smart” Policy

We don't have to pick a single direction for all states. Consider: go Left in states 2, 3, or 4, but go Right in state 5. States 1–4 keep the Case-A returns (100, 50, 25, 12.5). State 5 instead moves right to State 6: Return = $R(5) + \gamma R(6) = 0 + (0.5 \times 40) = 20$ — better than the 6.25 obtained by walking all the way left. This mixed policy achieves the highest possible return from every single starting state compared to Case A or Case B.



The discounted value of a future $100 reward, k steps out, for three different γ values. Smaller γ makes the agent impatient; γ close to 1 makes it far-sighted.

Pushing Negative Rewards Into the Future

Because $\gamma < 1$, a negative reward in the future is discounted heavily. Paying $10 today is a reward of –10; postponing that payment by a few steps discounts its impact, making it "less bad" in the eyes of the optimization algorithm. Consequently, the discount factor mathematically forces the RL agent to **push negative rewards as far into the future as possible** — exactly the behavior we want when delaying a crash or deferring costs.

7. The Policy π

Definition

A policy, denoted π (pi), is a function that maps any input state *s* to an action *a*:

$$\pi(s) = a$$

The Controller View

While "policy" is the standard convention in RL literature, calling it a **controller** is often much more intuitive — it acts as the brain or autopilot of the system, constantly reading the environment state and outputting control signals.

The Goal of RL

Find a policy π that **maximizes the expected return** from any starting state.

8. Markov Decision Process (MDP) Formalism

All the concepts developed through the Mars Rover example compose a formal mathematical framework called a Markov Decision Process (MDP).

The Markov Property

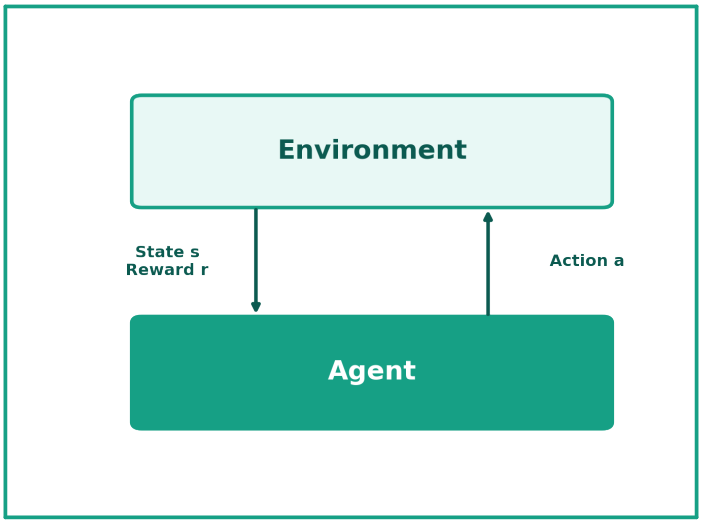
> *"The future depends only on the current state, and not on anything that occurred prior to reaching it."*

In other words, the history of how the agent arrived at state *s* contains zero predictive value for what happens next; the current state *s* is a sufficient statistic of the environment.

The Agent-Environment Interaction Loop

An MDP is visualized as an elegant feedback loop between two primary entities:

- **The Agent** — the decision-maker, controlled by the policy π.
- **The Environment** — the world in which the agent operates.



The agent observes state s and reward r, then selects action a; the environment responds with a new state s′ and reward r′.

The agent observes the current state *s* and receives a reward *r*, then selects an action *a* according to its policy π(s). The environment processes the action, changes its internal state, and emits a new state *s′* and new reward *r′* back to the agent.

Part 3 — Interview Prep & Real-World Application

Topic / Concept	ML Interview Question	Core Takeaway	Industry Application
Supervised Learning vs. RL	Why can't we easily fly an autonomous helicopter using standard Supervised Learning on expert telemetry? — Continuous physical control is highly ambiguous; there is rarely a single "ideal" label y for a given state x.	Supervised learning maps inputs to labels via explicit demonstrations (x → y). RL maps states to actions by maximizing a scalar reward (s → a), instructing the agent *what* to achieve rather than *how*.	**Autonomous Warehouse Robotics**: instead of programming joint angles for every package shape, RL rewards successful pick-and-place, letting the robot discover its own trajectory strategies.

Topic / Concept	ML Interview Question	Core Takeaway	Industry Application
The Return & Discounting	What behavior does changing γ from 0.99 to 0.1 induce? — $\gamma = 0.1$ makes the agent extremely "myopic," prioritizing immediate rewards. $\gamma = 0.99$ makes it far-sighted, willing to endure zero/negative rewards to reach a large payoff.	The Return is the discounted sum of future rewards. $\gamma \in [0, 1)$ models impatience, ensures convergence over infinite horizons, and forces the postponement of negative rewards.	**Algorithmic Stock Execution**: an RL agent balances the risk of market volatility (delaying sales) against the market-impact cost of dumping shares immediately.
Markov Decision Processes	Explain the Markov Property and why it's computationally crucial. — Future transitions and rewards depend only on the current state s, not the historical trajectory, which lets algorithms discard history and stay tractable.	An MDP formalizes RL via the tuple: States, Actions, Transitions, Rewards, and a Discount Factor. The current state must contain all information needed to predict the next state and reward.	**Dynamic Factory Resource Allocation**: a factory's state (machine queues, operational status) is framed as an MDP to optimize processing throughput.

Part 4 — Tailored Report (For the ML-Foundations Learner)

Lecture Overview & Specialization Context

This lecture marks your official transition into Reinforcement Learning, the final pillar of the Machine Learning Specialization.

- **Where it fits**: You have focused on Supervised Learning (explicit labels y) and Unsupervised Learning (patterns in unlabeled data). RL is a paradigm shift where the model learns through active interaction and trial-and-error feedback.
- **What's next**: This lecture establishes the conceptual scaffolding. Upcoming lectures move from *calculating* returns manually to *learning* them computationally using State-Action Value functions (Q-learning) and Deep Q-Networks (DQNs).

Foundational vs. Supplementary Mapping

Foundational (must master to proceed)	Supplementary / Context (non-blocking)
MDP Formalism (States, Actions)	Real helicopter dimensions and physical sensor suites
The Return Formula & Gamma	Specific details of robotic quadruped leg dynamics
Policy Mapping ($\pi(s) = a$)	
The Markov Property	

Commonly Tricky Areas

1. **Reward Timing Confusion**: Beginners often assume the reward R(s) is received *after* the action transitions you to the next state s′. Under the standard formalism, R(s) is strictly associated with the state s you currently occupy at the start of the decision step.

2. **The Trap of $\gamma = 1$**: It's tempting to remove discounting to treat all rewards equally. But in tasks without a guaranteed terminal state (infinite horizon), $\gamma = 1$ causes the Return to sum to infinity, breaking the optimization objective. Discounting is a mathematical necessity, not just a modeling choice.

Concrete Action Items for Mastery

1. **Pencil-and-Paper Derivation**: Re-derive the Mars Rover return values starting from States 2, 3, 4, and 5 under the "Always Go Left" and "Always Go Right" policies with $\gamma = 0.5$. Your numbers should match 50, 25, 12.5 and 10, 20, 5 exactly.
2. **Mental Mapping**: Take a simple environment you use daily (a web browser, a thermostat) and sketch its MDP mapping — what is the State S? The Actions A? A reasonable Reward Function R?
3. **Lab Prep**: Read through the setup for the upcoming Lunar Lander simulation lab. Identify the continuous state variables (coordinates, angles, velocities) and the discrete action set (engine firings) to prepare for your first hands-on RL implementation.

Takeaway: Reinforcement Learning formalizes sequential decision-making as an agent-environment loop governed by an MDP: a set of **states**, **actions**, and a **reward function** R(s) that tells the agent *what* to achieve, not *how*. The **Return** $G_t = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$ discounts future rewards by γ, encoding impatience and pushing negative rewards into the future. The **Policy** $\pi(s) = a$ is the controller the agent seeks to optimize so that expected return is maximized from any starting state — the scaffolding for everything from Q-learning to Deep Q-Networks that follows.